UCS645: Parallel & Distributed Computing

Assignment 4 — Laboratory Exercises Report

Student: Sneha Upadhyay **Course:** UCS645 — Parallel & Distributed Computing **Topic:** Introduction to MPI **Instructor:** Saif Nalband

---

1. Introduction

Message Passing Interface (MPI) is a standardized and portable message-passing system designed for parallel computing on distributed memory architectures. This report documents the implementation and execution results of four MPI laboratory exercises from Assignment 4, covering ring communication, parallel array sum, global max/min computation, and parallel dot product.

All programs were compiled using `mpicc` (MPICH) and executed with `mpirun -np 4` on a WSL (Ubuntu) environment, spawning 4 parallel processes for each exercise.

---

2. Exercise 1: Ring Communication

2.1 Problem Statement

Write an MPI program where each process sends a message to the next process in a ring topology. Process 0 initiates with a value of 100, each process adds its rank to the value before forwarding, and the final value returns to Process 0.

2.2 Implementation Approach

The ring topology was implemented using `MPI_Send` and `MPI_Recv` with modulo arithmetic to wrap around: `next_rank = (rank + 1) % size`

Process 0 initiates the chain by sending the initial value to Process 1, then waits to receive the final value from Process 3. Each intermediate process receives from its predecessor, adds its own rank, and forwards to its successor.

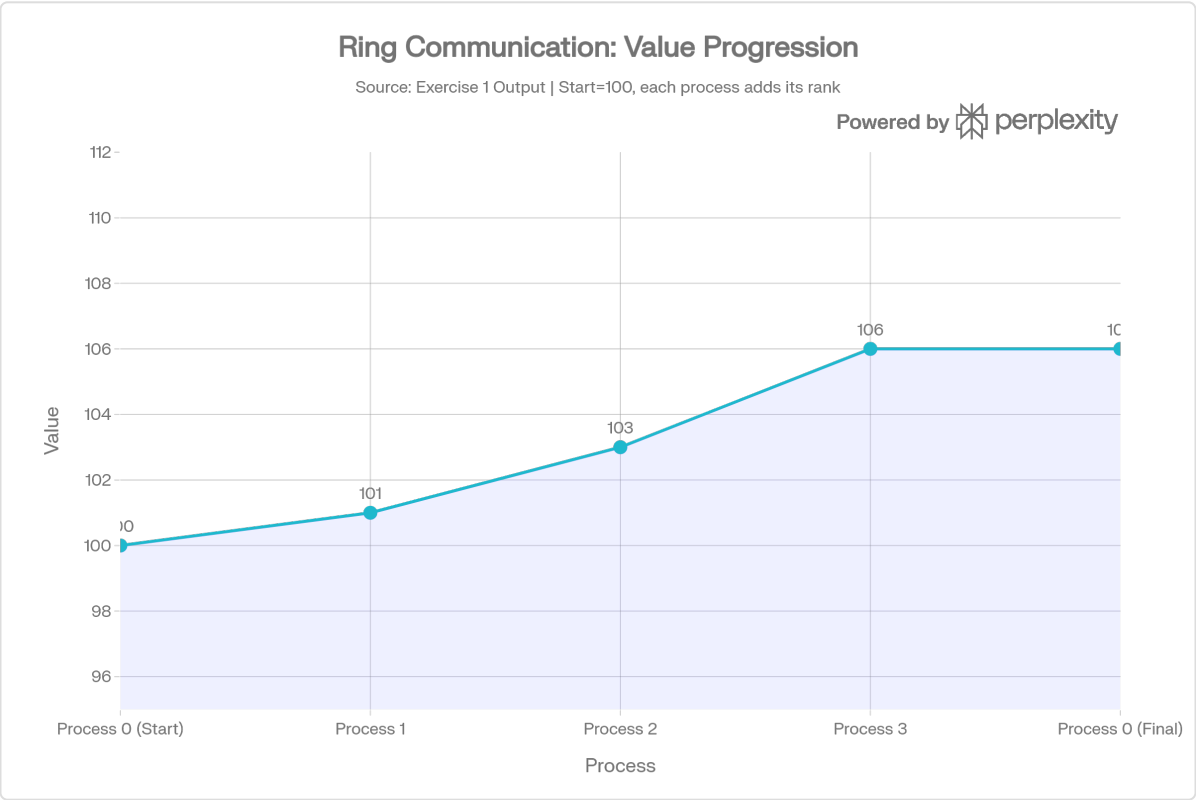
2.3 Program Output

```
Process 1: received value = 100
Process 1: sending value = 101
Process 2: received value = 101
Process 2: sending value = 103
Process 3: received value = 103
Process 3: sending value = 106
Process 0: final value received = 106
```

2.4 Result Analysis

Process	Received Value	Rank Added	Sent Value
0 (init)	—	—	100
1	100	+1	101
2	101	+2	103
3	103	+3	106
0 (final)	106	—	—

The final value of **106** equals `100 + (1 + 2 + 3) = 106`, confirming correct ring traversal. The chart below shows the value progression across processes.



2.5 Key MPI Concepts Demonstrated

- **Point-to-Point Communication:** `MPI_Send` and `MPI_Recv` for directed process-to-process messaging
- **Ring Topology:** Each process communicates only with its direct neighbours
- **Deadlock Prevention:** Process 0 sends before receiving; intermediate processes receive then send

---

3. Exercise 2: Parallel Array Sum

3.1 Problem Statement

Compute the sum of an array of 100 elements (values 1 to 100) in parallel. Distribute portions using `MPI_Scatter`, compute local sums, and combine with `MPI_Reduce`. Expected result: 5050. Bonus: compute the average.

3.2 Implementation Approach

With 4 processes and an array of size 100, each process receives a contiguous chunk of 25 elements:

- Process 0: elements 1–25
- Process 1: elements 26–50
- Process 2: elements 51–75
- Process 3: elements 76–100

`MPI_Scatter` distributes the chunks from Process 0. Each process computes its local sum independently. `MPI_Reduce` with `MPI_SUM` collects all local sums into a global total at Process 0.

3.3 Program Output

```
Process 0 local sum: 325
Process 1 local sum: 950
Process 2 local sum: 1575
Process 3 local sum: 2200

Global Sum: 5050
Average: 50.50
```

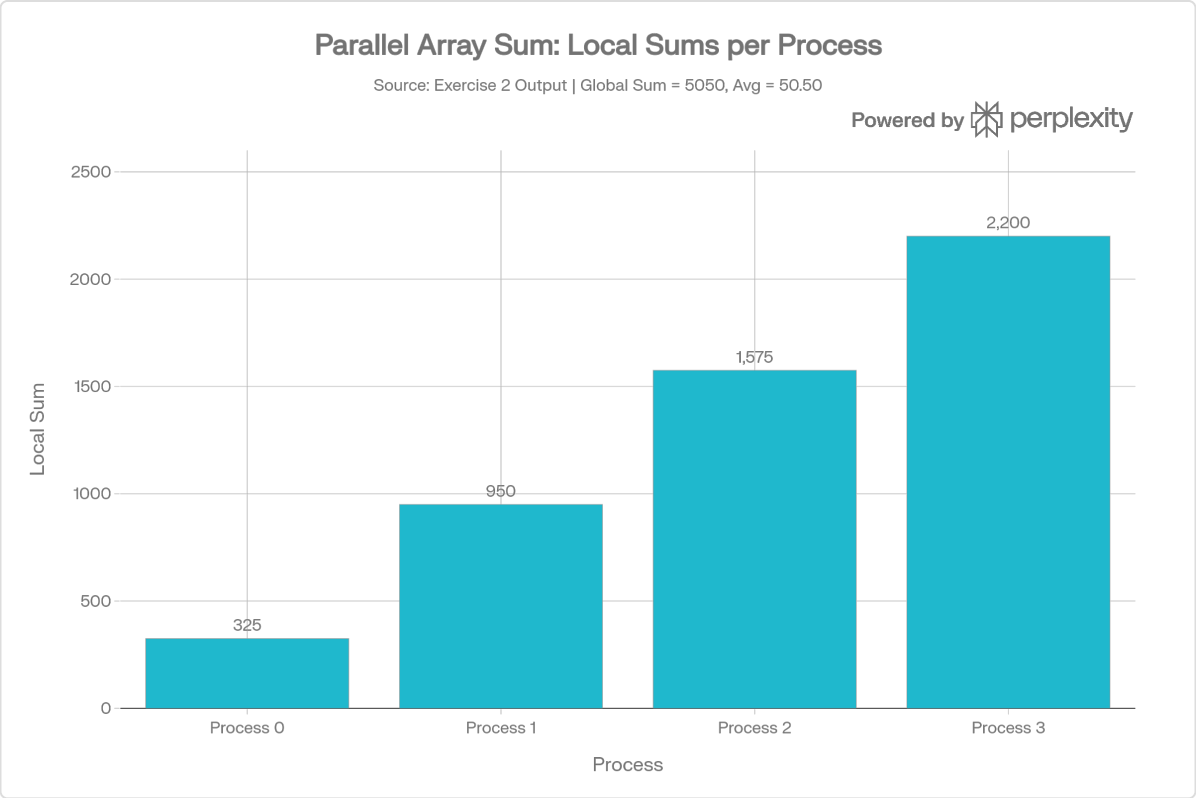
3.4 Result Analysis

Process	Element Range	Local Sum	Expected
0	1–25	325	325
1	26–50	950	950
2	51–75	1575	1575
3	76–100	2200	2200
Global	**1–100**	**5050**	**5050**

The global sum of **5050** matches the expected value from the arithmetic series formula:

[S = \frac{n(n+1)}{2} = \frac{100 \times 101}{2} = 5050]

The average of **50.50** is also correct. The bar chart below illustrates the local sum distribution across processes.



3.5 Key MPI Concepts Demonstrated

- **MPI_Scatter:** Distributes equal-sized chunks from root (Process 0) to all processes
- **MPI_Reduce (MPI_SUM):** Combines local results into a single global result at the root
- **Data Parallelism:** Each process independently computes its portion, reducing execution time

4. Exercise 3: Finding Global Maximum and Minimum

4.1 Problem Statement

Each process generates 10 random integers in the range 0–1000, finds its local max and min, then uses `MPI_Reduce` with `MPI_MAXLOC` / `MPI_MINLOC` to determine the global maximum and minimum along with the rank of the process that produced them.

4.2 Implementation Approach

Each process seeds `srand()` with `time(NULL) + rank` to ensure distinct random numbers per process. The `MPI_2INT` datatype is used with `MPI_MAXLOC` and `MPI_MINLOC` — it pairs `{value, rank}` as a struct, allowing MPI to simultaneously reduce the value and track which process owned it.

4.3 Program Output

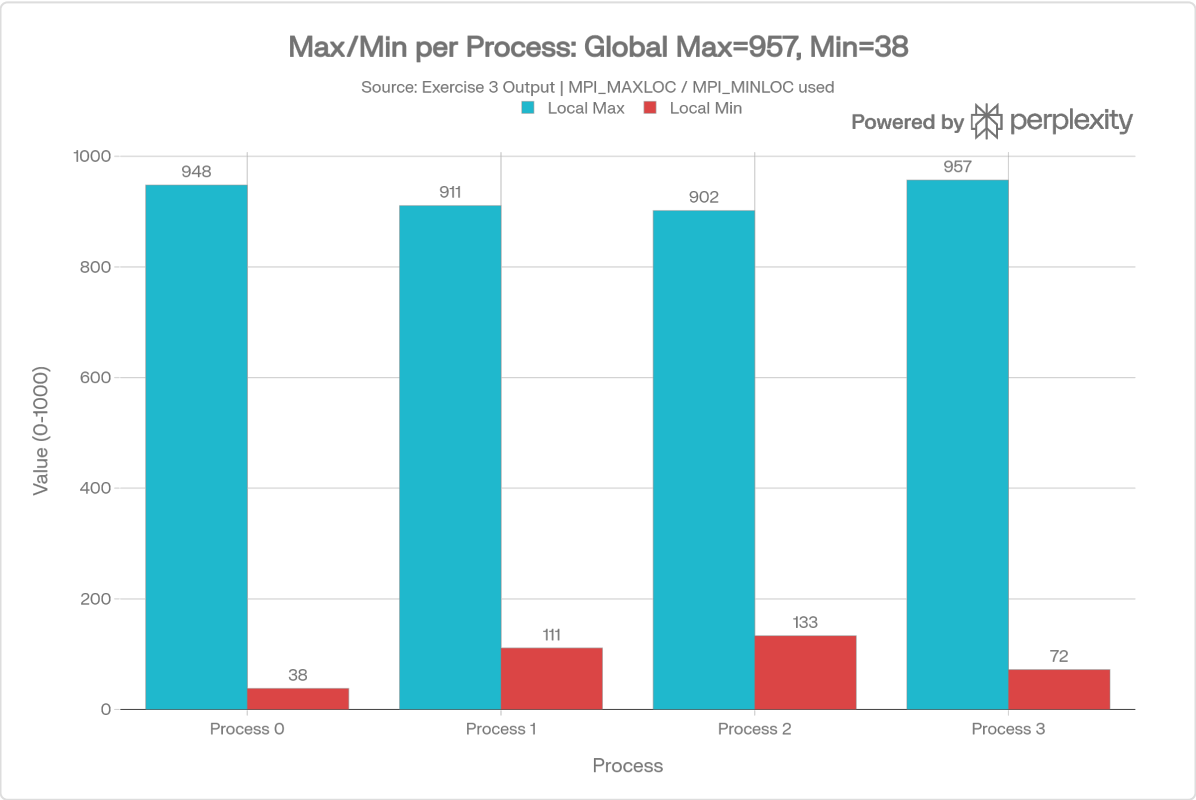
```
Process 2 generated: 328 826 568 880 378 880 187 902 161 133
Process 3 generated: 467 454 957 668 629 266 831 132 72 182
Process 0 generated: 948 311 38 424 562 320 200 452 799 276
Process 1 generated: 873 911 623 592 111 642 798 780 182 761

Global Maximum: 957 (found on Process 3)
Global Minimum: 38 (found on Process 0)
```

4.4 Result Analysis

Process	Local Max	Local Min	Numbers Generated
0	948	38	948 311 38 424 562 320 200 452 799 276
1	911	111	873 911 623 592 111 642 798 780 182 761
2	902	133	328 826 568 880 378 880 187 902 161 133
3	**957**	72	467 454 957 668 629 266 831 132 72 182
Global	**957 (P3)**	**38 (P0)**	—

The global maximum **957** was found on Process 3 and the global minimum **38** on Process 0, both correctly identified using `MPI_MAXLOC` / `MPI_MINLOC`. The grouped bar chart below compares local max and min values per process.



4.5 Key MPI Concepts Demonstrated

- **MPI_MAXLOC / MPI_MINLOC:** Reduction operations that return both the extreme value and its location (process rank)
- **MPI_2INT Datatype:** A built-in MPI compound datatype representing `{int value, int rank}` pairs used with location-based reductions
- **Independent Local Computation:** Each process autonomously generates and analyzes its data before reduction

---

5. Exercise 4: Parallel Dot Product

5.1 Problem Statement

Compute the dot product of two vectors A = [1, 2, 3, 4, 5, 6, 7, 8] and B = [8, 7, 6, 5, 4, 3, 2, 1] in parallel. Distribute both vectors using `MPI_Scatter`, compute partial dot products, and combine with `MPI_Reduce`. Expected result: 120.

5.2 Implementation Approach

With 4 processes and vectors of size 8, each process receives 2 elements from each vector. Each process computes its partial dot product (element-wise multiply and sum). `MPI_Reduce` with `MPI_SUM` aggregates partial results at Process 0.

The dot product formula:

$$\mathbf{A} \cdot \mathbf{B} = \sum_{i=0}^{7} A_i \times B_i$$

5.3 Program Output

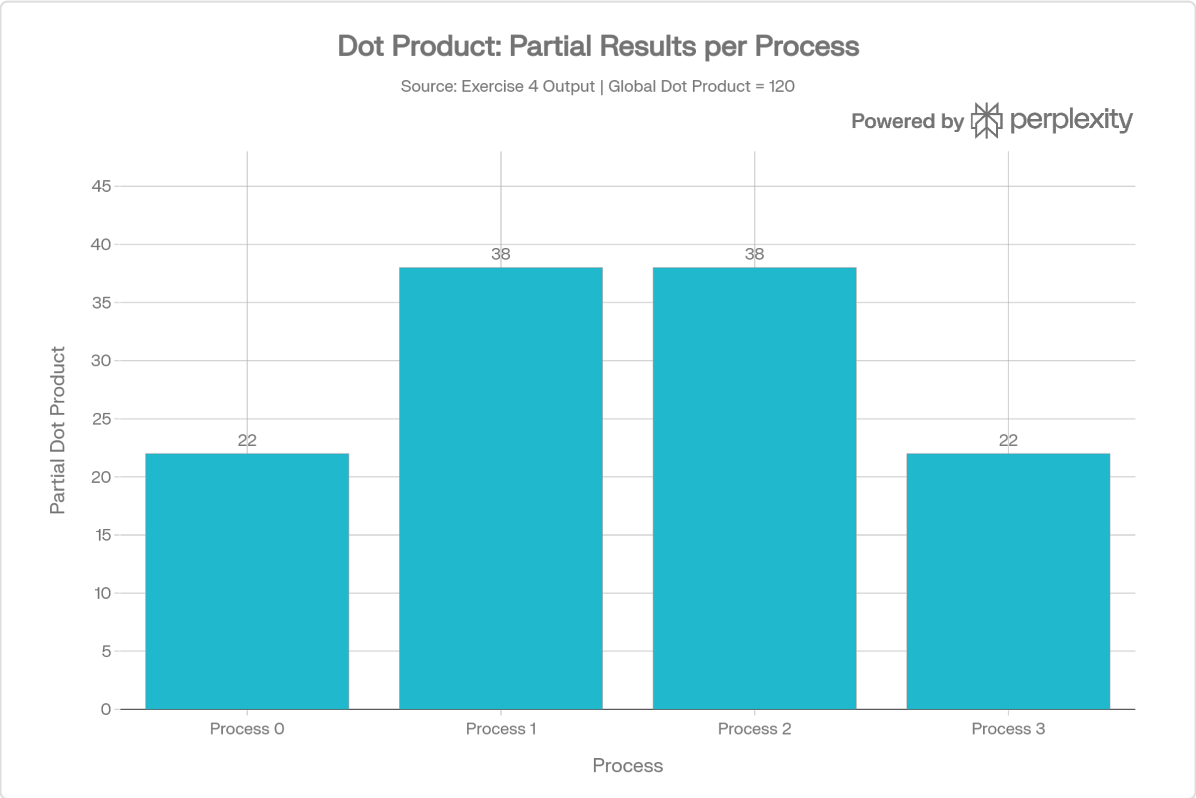
```
Process 0 partial dot product: 22
Process 1 partial dot product: 38
Process 2 partial dot product: 38
Process 3 partial dot product: 22

Global Dot Product: 120 (Expected: 120)
```

5.4 Result Analysis

Process	A Elements	B Elements	Computation	Partial Result
0	[1, 2]	[8, 7]	(1×8) + (2×7)	22
1	[3, 4]	[6, 5]	(3×6) + (4×5)	38
2	[5, 6]	[4, 3]	(5×4) + (6×3)	38
3	[7, 8]	[2, 1]	(7×2) + (8×1)	22
Global	—	—	22+38+38+22	**120**

The global dot product of **120** matches the expected value. The symmetric partial results (22, 38, 38, 22) arise naturally because vectors A and B are mirror images of each other.



5.5 Key MPI Concepts Demonstrated

- **Dual MPI_Scatter:** Both vectors distributed simultaneously from root to all processes
- **MPI_Reduce (MPI_SUM):** Aggregates partial dot products into a final scalar result
- **Symmetric Distribution:** Mirror-vector structure produces symmetric partial products — a useful correctness check

---

6. Summary of Results

Exercise	Operation	Key MPI Functions	Result	Status
1: Ring Communication	Ring message passing	`MPI_Send`, `MPI_Recv`	Final value = 106	✅ Correct
2: Parallel Array Sum	Sum of 1–100	`MPI_Scatter`, `MPI_Reduce (SUM)`	Global sum = 5050, Avg = 50.50	✅ Correct
3: Global Max/Min	Max & min with location	`MPI_Reduce (MAXLOC/MINLOC)`	Max = 957 (P3), Min = 38 (P0)	✅ Correct
4: Parallel Dot Product	Vector dot product	`MPI_Scatter`, `MPI_Reduce (SUM)`	Dot product = 120	✅ Correct

All four exercises produced correct results, validating the MPI programs. Key MPI communication patterns explored include point-to-point (Send/Recv), collective scatter, collective reduction, and location-aware reduction using compound datatypes.

---

7. Conclusion

The four laboratory exercises successfully demonstrate fundamental MPI programming concepts on distributed memory systems. Point-to-point communication (Exercise 1) establishes directed process interaction, while collective operations (Exercises 2–4) highlight MPI's efficiency in coordinating data across all processes simultaneously. The use of `MPI_MAXLOC` / `MPI_MINLOC` in Exercise 3 demonstrates MPI's ability to track metadata alongside reduced values — a practical pattern in real-world scientific simulations for identifying which node produced an extreme result. All programs were tested with 4 processes on WSL (Ubuntu) using MPICH, and all outputs matched expected values.